2026 年春季学期

数据结构课程设计实验报告

万阳 何俊翰

软件学院 2024 级 2 班

1 所选赛道

赛道 6 ：中国象棋(ChineseChess)

2 分工与合作

分工内容：

万阳(70%)：

中残局阶段权重计算；
棋子位置价值表/静态局面评估、
棋盘控制力统计/动态局面评估；
攻击机会、受威胁状态、重要棋子威胁检测；
特殊棋型评估；
着法预排序；
开局库 Zobrist hash 命中算法及 Zobrist-64 hash 记录算法；
Transposition Table(置换表) Exact,Alpha,Beta 节点记录与启发搜素；
NegaMax(负极大值)形式 alpha-beta 搜素算法；
Fail-Soft 形式 PVS(Principal Variation Search，主要变例搜素)算法；
Null Move Pruning(空步裁剪)优化；
杀手着法启发排序优化
LMR(Late Move Reductions，晚期着法裁剪)优化；
Quiet Moves 安静着法优化

何俊翰(30%)：

MVV/LVA (最大价值受害者/最小价值攻击者)评价；
着法预排序；
Quiescence Search(静态搜素)算法；
搜素时间控制与状态保护；

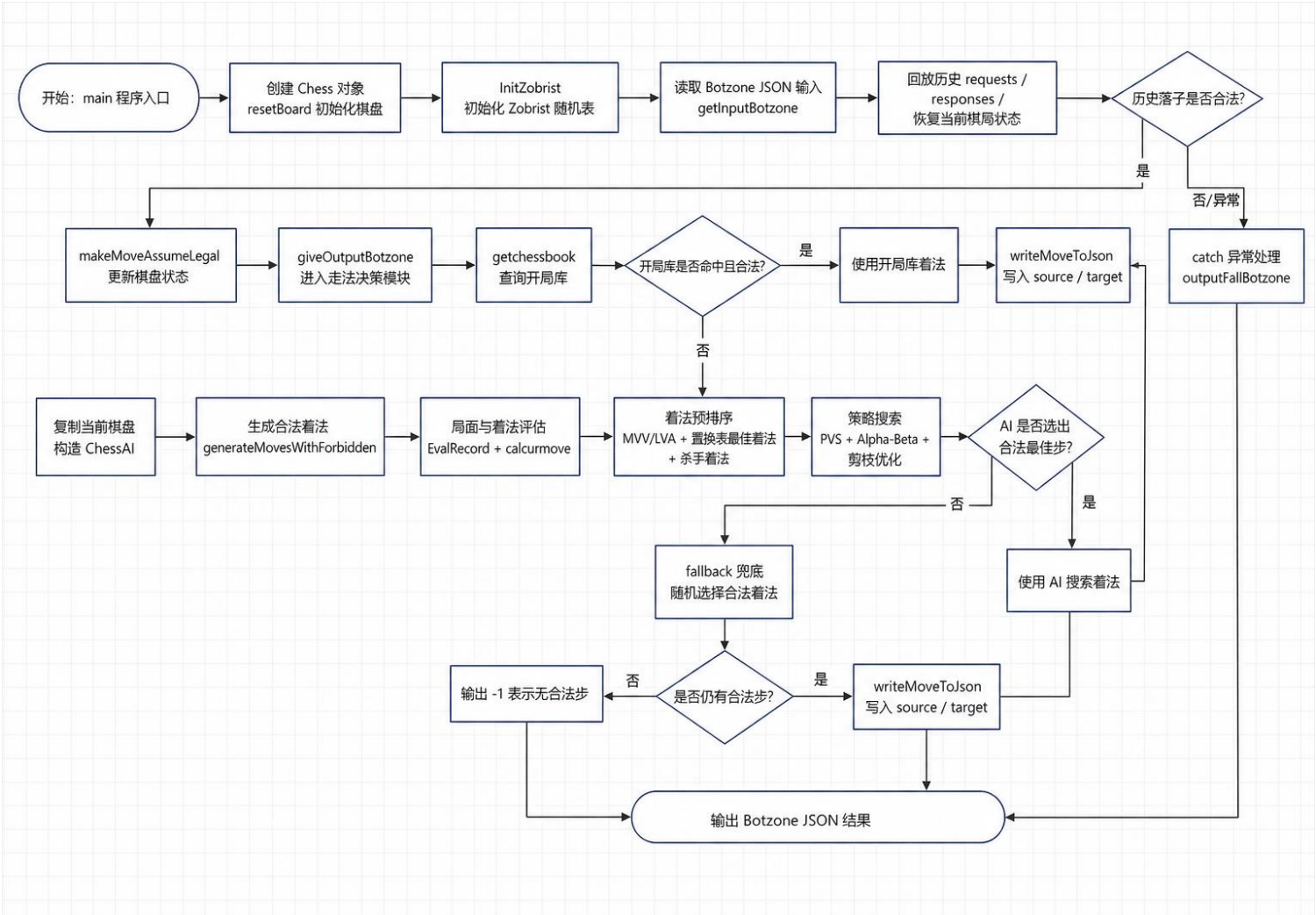
延迟选择排序；
不良吃子剪枝；
将军状态的特殊处理；
将程序转换为 rust；

3 算法思想

3.1 总体思路

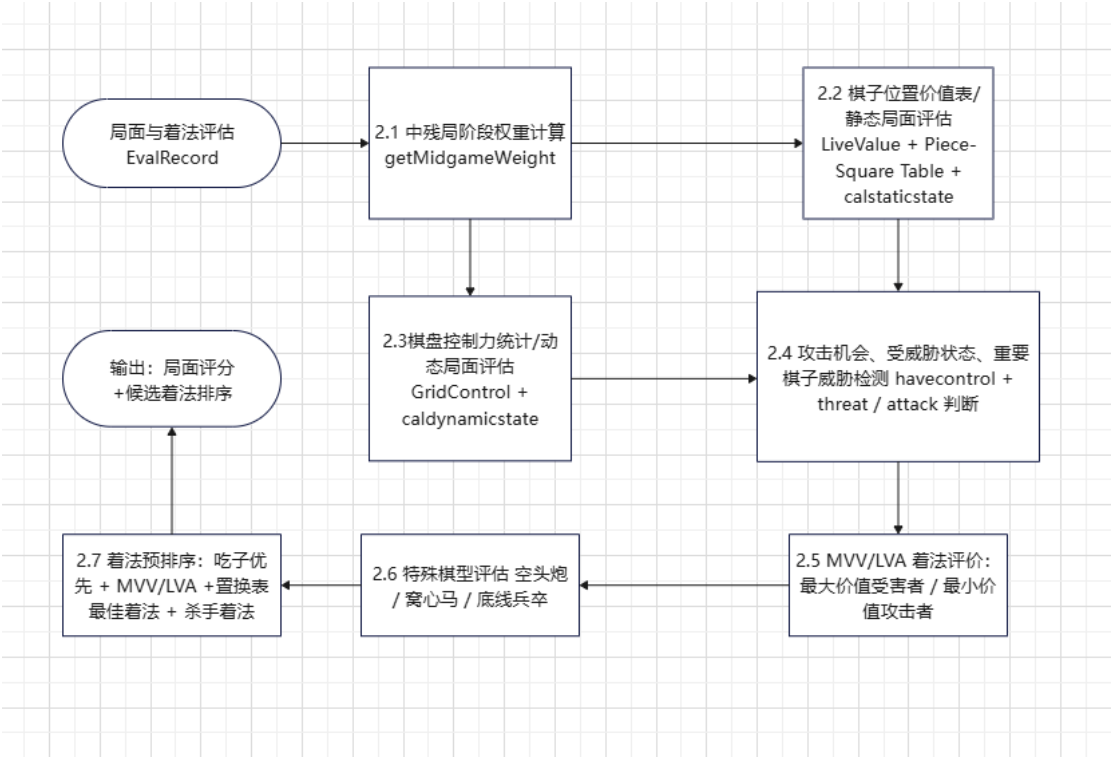
本程序的总体思想是将中国象棋的自动着棋的评测问题，转化为“合法着法生成——局面价值评估——博弈树搜素——最优着法筛选”的决策过程。

程序整体的流程图如图所示：



程序首先根据评测平台输入的历史走法恢复当前棋局状态，然后在当前局面下生成所有满足中国象棋规则的合法着法；在此基础上，对每个候选着法产生的新局面进行评估，并通过搜索算法向后推演若干层博弈树，最终选择在当前搜索深度和时间限制内评估值最高的着法作为输出。

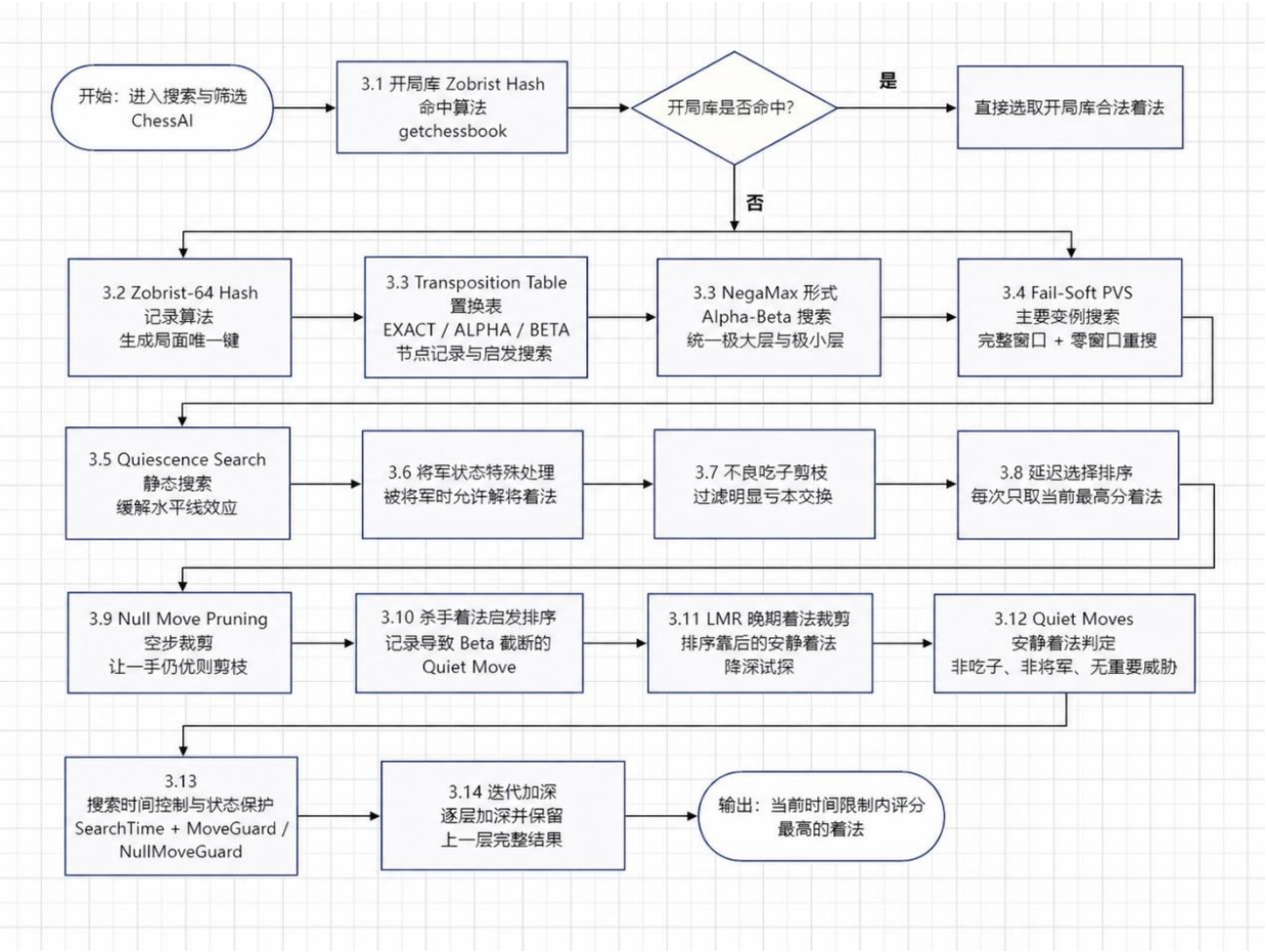
程序第二部分(局面价值评估)的流程图如图所示：



本程序的局面评估部分由 EvalRecord 类集中实现，其核心思想是：不只根据双方剩余子力判断优劣，而是综合考虑棋子价值、棋子位置、棋盘控制力、攻防关系、特殊棋型和当前棋局阶段，得到一个更接近真实棋势的综合评分。

EvalRecord:包括 getMidgameWeight、calstaticstate、caldynamicstate、calShapePenalty、calstate 等函数，分别用于中残局阶段判断、静态估值、动态估值、棋型惩罚和总局面评分。

程序第三部分(策略搜素与着法筛选)的流程图如下图所示：



本程序的搜索剪枝部分由 ChessAI 类实现，其核心目标是：在 Botzone 有限的时间内尽可能向更深层搜索，同时通过排序、缓存和剪枝减少无效分支，使程序更快找到较优着法。

搜索部分以 NegaMax 形式的 Alpha-Beta 为基础，并进一步加入 PVS、置换表、静态搜索、空步裁剪、LMR、杀手着法、迭代加深和时间控制等优化。ChessAI 中定义了 alpha_beta_PVS_select、alpha_beta_PVS_search、quiescence_search、isQuietMove、killerMoves 等搜索相关结构和函数。

本程序的总体框架可以概括为：先由规则层保证着法合法，再由评价层估计局面优劣，最后由搜索层在候选着法中选择收益最大的方案。开局阶段尽量利用开局库快速落子；中后盘阶段则依赖局面评估函数和多种剪枝优化后的博弈树搜索，在搜索质量与运行时间之间取得平衡。

3.2 所用方法

程序总体分为三部分(第一部分接口与方法由评测程序给出，第二、三部分为主要内容)：

一、着法生成与合法性检查(大部分内容由 Botzone 示例评测程序给出)

二、局面与着法评估，其中包含以下部分：

2.1 中残局阶段权重计算

方法描述：在 EvalRecord::getMidgameWeight 中遍历当前棋盘，根据棋子类型累计阶段权重。其中车、马、炮等大子较其它普通棋子对中局贡献更高，普通棋子贡献较低；并通过二次插值将局面映射到 0 到 Total_state 之间的中局权重，被 calstateicstate 调用用于过渡计算静态位置得分。

2.2 棋子位置价值表/静态局面评估

方法描述：在 namespace Evalpara 中定义各类棋子的基础存活价值 LiveValue、将帅安全表 KingSafeTable、士位置表 AdvisorPTable、相位置表 BishopPTable、兵卒中/残局位置表 PawnMidPtable/PawnEndPTable。通过 EvalRecord::calstaticstate 遍历棋盘，并调用 staticPieceValue 计算基础子力价值+位置修正值，按照己方加分、敌方减分的方式得到静态局面评分

2.3 棋盘控制力统计/动态局面评估

方法描述：在 EvalRecord::getGridcontrol 中遍历棋盘所有棋子，并按照象棋规则统计红黑双方对 90 个格点的控制次数，存入 GridControl 数组。EvalRecord::caldynamicstate 再根据控制点位于敌方九宫、敌方半场、己方九宫或己方半场的不同位置赋予不同权重，同时结合进攻对方有子点、保护己方有子点、无根子惩罚等因素，计算双方控制力差值作为动态局面评分。

2.4 攻击机会、受威胁状态、重要棋子威胁检测

方法描述：在 EvalRecord::getGridcontrol 中统计全局格点控制权，并填充 havecontrol 数组。在搜索逻辑中，havethreat_of_valuestone 函数专门用于检测重要棋子是否受攻，其结果直接影响 isQuietMove 的判定（判定该步是否为“安静着法”），进而指导 LMR 裁剪和空步裁剪的执行。

2.5 MVV/LVA (最大价值受害者/最小价值攻击者)评价

方法描述：在 Tuning::getMVVLVAScore 中针对象棋搜索树中的吃子着法进行优先级评估，主要计算“最有价值受害者（MVV）”与“最无价值攻击者（LVA）”之间的交换收益。程序首先检查着法是否为吃子，若存在被吃子则赋予固定的基础高分（CaptureBaseScore），确保吃子着法绝对优先；再提取被吃子的静态价值并乘以 10 作为奖励得分；若被吃子处于敌方保护状态，则通过 isProtected 标识触发 LVA 机制，直接扣除攻击子自身的静态价值作为反吃惩罚，分别识别出“小子换大子”的高收益形态或“大子换受保护小子”的亏本形态。最终将上述各项分值相加得出该着法的综合评分，在静态搜索的着法排序中提供精确的优先级依据，极大提升剪枝效率。

2.6 特殊棋型评估

方法描述：在 EvalRecord::calShapePenalty 中针对象棋中的特殊危险棋型进行额外惩罚，主要检测空头炮、窝心马以及底线兵/卒等情况。程序首先判断我方将帅是否处于固定底线位置，再检查中路是否存在中门空虚；若敌方炮位于中路，则通过 betweenNotEmptyNum 统计炮与将帅之间的阻挡子数量，分别识别空头炮或以窝心马为炮架的危险形态，并根据距离查表加入罚分。最终在 calstate 中将己方棋型罚分扣除、对方棋型罚分加回，修正 calstate 集成评分方法。

2.7 着法预排序：吃子优先、MVV/LVA、置换表最佳着法、杀手着法

方法描述：在 EvalRecord::calcurmove 中对每一步着法进行初步打分，吃子着法使用 MVV/LVA 分值，非吃子着法则使用移动棋子的基础价值和推进奖励。随后 fill_sort_inMoveNodes 将 Move 转换为 MoveNode 并调用 MoveNodesort 稳定降序(非升序)排列。在 PVS 搜索内部，若置换表中存在 best_move，则优先将该着法交换到首位；之后再将当前深度记录的杀手着法提前，从而形成“吃子/MVV-LVA → 置换表最佳着法 → 杀手着法”的综合预排序顺序，提高 alpha-beta 剪枝效率。

三、策略搜素和着法筛选，其中包含以下部分

3.1 开局库 Zobrist hash 命中算法

方法描述：程序通过 InitZobrist 初始化固定随机种子的 ZobristTable 和 ZobristPlayer，并在 Chess::getZobristKey 中遍历棋盘，将每个非空棋子的“类型、颜色、位置”对应特定 key 值(程序中设为 20262026 Unsigned long long)进行异或，若当前为黑方行动则额外异或 ZobristPlayer，从而得到包含行棋方信息的 64 位局面键。getchessbook 使用该键转换成字符串后查询 data/opening_book.json，在命中开局库时选取出现次数最高的候选着法；若多个着法出现次数相同，则随机选择其中之一，并在输出前再次调用 isMoveValidWithForbidden 校验合法性。

3.2 Zobrist-64 hash 记录算法

方法描述：程序使用 uint64_t 表示棋盘局面的 Zobrist Key，通过对棋盘上所有棋子与特定 key 值(程序中设为 20262026 Unsigned long long)进行异或，把复杂棋盘状态映射为 64 位整数，计算得到的 Hash 值供开局库和置换表使用，该，保证局面重复识别与搜索记忆化。

3.3 Transposition Table(置换表) Exact,Alpha,Beta 节点记录与启发搜素

方法描述：程序定义 TTStateNode 记录 Zobrist Key、搜索深度、评估分数、节点类型和最佳着法，并使用固定大小 HashTable 存储搜索过的局面。probeHash 在进入搜索节点时根据 Zobrist Key 查询置换表，若命中且记录深度不低于当前深度，则根据 EXACT、ALPHA、BETA 三种标志决定直接返回精确值或利用上下界进行剪枝。RecordHash 在搜索结束或发生 beta 截断时写入置换表，并优先保留更深层的搜索结果。同时，命中的 best_move 会在当前节点的着法排序中被提前，作为置换表启发着法优先搜索。

3.3 NegaMax(负极大值)形式 alpha-beta 搜素算法

方法描述：程序的 alpha_beta_search 与 alpha_beta_PVS_search 均采用 NegaMax 形式实现，即每向下一层搜索一次，就通过取负号转换为对手视角的局面评分，调用形式为 -search(depth - 1, -beta, -

alpha)。这样可以将极大层和极小层统一为“当前行动方最大化收益”的搜索模型，简化递归逻辑。搜索过程中若 val >= beta 则触发 beta 剪枝；若 val > alpha 则更新 alpha，最终返回当前节点在 alpha-beta 窗口内得到的最优估值。

3.4 Fail-Soft 形式 PVS(Principal Variation Search，主要变例搜索)算法

方法描述：在 ChessAI::alpha_beta_PVS_search 中实现主要变例搜索。第一个候选着法使用完整窗口搜索，之后的着法先使用零窗口 [-alpha-1, -alpha] 进行快速验证；若返回值落入 alpha 与 beta 之间，说明该着法可能改写主要变例，再使用完整窗口重新搜索。程序使用 best_val 保存当前节点实际搜索到的最优分数，并在 beta 截断时记录 BETA 节点，在搜索结束后根据 best_val 是否超过 original_alpha 记录 EXACT 或 ALPHA 节点，使 PVS 与置换表记录相结合，提高深层搜索效率。

3.5 Quiescence Search(静态搜素)算法

方法描述：在 ChessAI::quiescence_search 中针对博弈树深度截断导致的“水平线效应”进行延伸搜索，主要处理吃子和被将军等剧烈变化的局面。程序首先进行深度与时间保护，扫描九宫格，考虑将军状态是否需要特殊处理。若未被将军，程序会计算当前局面的静态基础

分（Stand Pat），若该分数大于等于 beta 则直接触发剪枝，否则尝试更新 alpha 下界。接着，程序生成所有合法步，仅将吃子着法或解将着法纳入搜索；通过 board->isattacked 判断目标位置保护状态以执行“不良吃子剪枝”（剔除亏本的交换行为），并利用 MVVLVA（最有价值受害者-最无价值攻击者）对着法进行高优先级打分。最终，程序采用延迟选择排序逐一提取当前最高分着法，递归调用带有深度递减的 quiescence_search 进行负极大值推演，结合软失败机制返回该节点下的最真实评估分数。

3.6 将军状态的特殊处理

方法描述：在 Quiescence Search 中只扫描九宫格找到己方将帅，并检测是否被将军（inCheck）。如果处于 inCheck，禁用 Stand Pat（。在后续着法生成时，如果不被将军，只搜索吃子着法（victim != None）；如果被将军，则允许所有合法着法（包括非吃子的逃将着法）进入静态搜索队列。在将军时，逃将着法（非吃子）被赋予 AttackerValue[attacker] 的分数，而吃子着法被赋予 mvvlva + 10000。这确保了在解将时，吃掉对方将军的子的优先级远远高于单纯解将。

3.7 不良吃子剪枝

方法描述：在 Quiescence Search 中剔除明显价值不对等的着棋步。在当前未被将军 (!inCheck)、目标子有保护 (isProtected)、受害者的价值低于攻击者的价值 (VictimValue < AttackerValue)时进行剪枝，避免导致搜索树过度膨胀而引发的连续交换；在例外时进行保护 ：if(Tuning::VictimValue[victim] < 3)（假设 3 代表马/炮的价值），这意味着，即使是价值不对等，只要吃的是大子，依然保留搜索，只有在“用大子换对方有保护的低价值子（如小兵）”时才执行剪枝（continue）。

3.8 延迟选择排序

方法描述：在 Quiescence Search 中遍历着法数组时，每次外层循环只在剩余未处理的着法中找出一个最高分的，找到后立刻执行搜索，因为 Alpha-Beta 搜索通常在测试第 1 个或第 2 个好着法时就会触发 val >= beta 剪枝并退出函数，此时延迟排序只有在需要挑出好着法时才进行排序操作，避免使用全局 std::sort 全排序在静态搜索中造成性能浪费。

3.9 Null Move Pruning(空步裁剪)优化

方法描述：在 alpha_beta_search 和 alpha_beta_PVS_search 中，当剩余深度满足 depth >= R + 1 且当前局面不存在明显受威胁状态时，程序执行一次空步，即通过 makeNullMove 直接交换行棋方而不移动棋子。随后使用缩减深度 depth - 1 - R 和零窗口进行搜索；若空步后的返回值仍然不低于 beta，说明当前局面对己方优势足够大，即使让一手也不会跌出 beta 窗口，因此可直接返回 beta 形成剪枝。执行完空步后通过 undoNullMove 恢复局面，保证搜索树状态正确。

3.10 杀手着法启发排序优化

方法描述：程序在 ChessAI 维护 killerMoves[max_depth][2]，为每个搜索深度保存两个导致 beta 截断的非吃子安静着法。在 PVS 搜索中，如果某个 Quiet Move 触发 val >= beta，则将其记录为当前深度的杀手着法，并把原第一杀手着法顺延到第二位置。后续同深度节点生成着法后，会在置换表最佳着法之后检查杀手着法，并将命中的着法提前到搜索列表前部，使曾经造成剪枝的强制性着法更早被搜索。

3. 11 LMR(Late Move Reductions，晚期着法裁剪)优化

方法描述：在 alpha_beta_PVS_search 中，程序对排序靠后的安静着法进行 LMR 优化。当已经找到主要变例 foundPV、当前着法不是第一个着法、搜索深度不小于 3，且 isQuietMove 判定该步不是吃子、不是将军、也不制造重要棋子威胁时，先用减少 1 层深度的零窗口搜索进行试探。我们断言：由于这种着法在浅层没有引起剧烈变动，它很可能是一种极其平庸的着法，即缩减结果预期小于 alpha。如果该缩减搜索结果仍然大于 alpha，说明该着法可能被低估，再恢复完整深度并使用全窗口重新搜索。通过以上方式，减少大量低价值后续分支。

3.12 安静着法 Quiet Moves

方法描述：在 ChessAI::isQuietMove 中对候选着法进行安静性判定。程序首先排除非法坐标、源格非当前行动方棋子以及所有吃子着法；随后使用 MoveGuard 临时执行该步，并检查该步是否造成将军、是否制造对车马炮等重要棋子的威胁。若该步既不吃子、不将军，也不产生重要战术威胁，则认为是安静着法。该结果被 LMR 用于判断是否可以降低搜索深度，也被杀手着法逻辑用于只记录非吃子的高价值剪枝着法。

3.13 搜素时间控制与状态保护

方法描述：在 SearchTime::check 中针对 AI 博弈树搜索的单步耗时进行控制与阻断，主要防止引擎超时。程序在搜索起步时通过 steady_clock 设定未来的超时截止点（endTime）。在搜索递归的深层节点中，引擎会高频调用内联的 check 方法；用无符号整型计数器（checkerCount）与位与运算（& 255）执行频率降级，使得每历经 256 个节点才实际比对一次物理时钟。一旦检测到当前时间已越过截止点，程序将立即抛出自定义的 TimeOutException 异常，通过 C++ 的异常处理机制迅速引发调用栈回溯（Stack Unwinding），瞬间终止所有正在深潜的搜索分支，实现计算时间硬截断。

3.14 迭代加深

方法描述：在 ChessAI::alpha_beta_PVS_select 中采用迭代加深策略，外层从深度 1 逐步搜索到 max_depth。每完成一层深度后，程序使用 stable_sort 按搜索得分重新排列根节点着法，并保存 last_completed_selected 作为当前已完整完成的最好结果。下一层搜索继承上一层排序结果，使更可能成为最佳着法的分支优先进入 PVS 搜索，从而提升剪枝效率。若搜索过程中超时，则捕获 TimeOutException，并恢复到上一层完整搜索结果；若发现分数超过必胜阈值(一个极大的值，程序中记为 900000)，则提前结束更深层搜索。

3.3 算法性能分析与测试结果

3.3.1 算法性能分析

主搜素时间复杂度

程序使用 PVS + Alpha-Beta + 置换表 + 空步裁剪 + LMR + 杀手着法。但严格最坏情况不能假设剪枝有效。

主搜素的迭代深度从 1 层到 10 层，层数 d 的搜素树节点大致为：

$$\sum_{i=1}^{d} B^i$$

总迭代加深节点数(D = 10)：

$$N_{\mathrm{ID}}(B,D)=\sum_{d=1}^{D}\sum_{i=1}^{d}B^i=\sum_{i=1}^{D}(D-i+1)B^i$$

其中

$$D=10$$

因此总程序最坏复杂度为：

$$T_{\mathrm{main}}=O\left(C_{\mathrm{node}}\sum_{i=1}^{10}(11-i)B^i\right)=O(C_{\mathrm{node}}B^{10})$$

程序空间复杂度分析：

程序空间复杂度与内存开销分析如下：

本程序的主要内存开销来自于置换表、Zobrist 表和棋盘状态和搜素内存。

置换表的结构体：

```
struct TTStateNode {
    uint64_t zobrist_key;
    int depth;
    int score;
    HashFlag flag;
    Move best_move;
};
```

在 Botzone 64 位 g++ 二进制接口中，有：

$$sizeof(Move) = 16$$

$$sizeof(TTStateNode) = 40$$

置换表内存为：

$$2^{20} \times 40 = 41943040 \text{ bytes} = 40 \text{ MiB}$$

这是程序最主要的固定内存开销。

Zobrist 表结构：

$$ZobristTable[8][2][90]$$

内存大小：

$$8 \times 2 \times 90 \times 8 = 11520 \text{ bytes} = 11.25 \text{ KiB}$$

递归搜素局部向量

主搜素 10 层，局部 vector 的严格上界：

$$10 \times 119 \times (16 + 20) = 42840 \text{ bytes} \approx 41.8 \text{ KiB}$$

静态搜素 15 层，每层有固定数组 $MoveNode\ qsNodes[128]$

严格上界

$$15 \times 2560 = 38400 \text{ bytes} \approx 37.5 \text{ KiB}$$

总空间复杂度约 40—50 MB，满足 Botzone 平台内存要求

3.3.2 测试结果




[东北雨姐]香港教材英文
[baizhitiao]bzt_3
游戏结束
红方胜利
当前回合：11
版权所有 © 2014 - 2026 北京大学信息科学技术学院 人工智能实验室
网站主要作者：zhf zhouhy zhq ...以及更多前辈
下载 Botzone 的对局数据集
京ICP备18028450号-1
中文 / Chinese


[baizhitiao]bzt_3
[JOEYCHOE]JOEYsecondbot
游戏结束
黑方胜利
当前回合：90
版权所有 © 2014 - 2026 北京大学信息科学技术学院 人工智能实验室
网站主要作者：zhf zhouhy zhq ...以及更多前辈
下载 Botzone 的对局数据集
京ICP备18028450号-1
中文 / Chinese





挑选的 5 局具有代表性的棋局如上所示，即与五轮 botzone 上排行榜较为靠前的 5 名对手的对战结果如上，3 胜 1 负 1 平，可以认为具备较为出色的对战能力，测试中途没有出现非法棋步、超时等情况，可以在各种走法中找出优势棋步。

在实际测试运行中发现象棋 AI 具有较为的进攻欲望，擅长于使用少数棋子牵制对方大量棋子，这与我们构建的分数评估体系密切相关，另外 AI 在一些情况下有“换子”倾向，用同等价值的棋子与对方交换只为多获得较少的位置评分，这在残局我方占据劣势的情况下是不明智的。经过分析应该是由于在主搜索时间耗尽需要其他快速算法寻找优势棋步时未能高效评估局面，需要引入更加完善复杂的机制判断局面决定是否“换子”

4.总结

4.1 问题解决与解决过程

遭遇问题、分析与思考问题、直至解决问题的过程贯穿算法与程序的设计、编写过程的始终。回顾本项目从零到完备的构建过程，大致，出现大大小小许多问题，大致可以分为算法问题与程序问题，总结为以下几点：

4.1.1 算法设计问题

如何选择应用的算法？本程序应该使用哪种算法最符合实际情况？针对算法选择的问题，我们查阅了相关的论文，并结合自身实际情况，以及综合考虑时间和课设实际目标要求，最终确定了使用最为经典也是最为可靠的博弈树搜素算法 —— alpha – beta 搜素算法。

针对以上思路，我们以 alpha-beta 算法为核心，并根据相关论文的各种算法加以补充本程序的主要算法。事实上，我们曾考虑过使用

高深的深度学习/强化学习/神经网络等算法。但是，考虑到我们仅仅是大二学生，对于上述机器学习类算法掌握并不全面 —— 我们也仅仅是在课下折腾过大模型和数据分析，相应理论和算法能力并不算过硬。就三四周的紧张项目编写过程来说，肯定是不如使用经典搜素算法 —— 它内容全面，已经成熟和完善，足以完成本课程设计的完整任务。

4.1.2 算法优化问题

如何优化当前算法？仅仅使用 alpha-beta 算法当然是不够的。

首先，本程序使用 PVS 主变例搜索。第一个着法使用全窗口搜索，后续着法先用零窗口 [-alpha-1, -alpha] 试探，若发现可能超过 alpha，再重新全窗口搜索。这能在着法排序较好时显著减少节点数，将朴素 alpha-beta 算法的分支因子 d 降低至 d/2。

并且，借鉴算法竞赛中“记忆化搜素”的思路：将已经搜素过的节点对应的估值放入哈希表中，即使用置换表。程序通过 Zobrist Hash 将局面映射到 HashTable，表项包含 depth、score、flag、best_move，并区分 EXACT、ALPHA、BETA 三种节点类型。命中置换表后，如果搜索深度足够且边界条件满足，可以直接返回，避免重复搜索。

迭代加深则参考学术论文中的方法。alpha_beta_PVS_select 从浅层逐步搜索到最大深度，每完成一层就排序根节点着法，并保存上一层完整结果，超时时回退到最后完整深度。这是比较合理的实时对弈设计。

水平线效应是棋类游戏 AI 引擎中经常出现的问题。为了避免水平线效应，我们使用 quiescence search：当 depth 为 0 时不直接调用静态评估，而进入静态搜索，继续搜索吃子等战术着法，以缓解水平线效应。

根据启发式搜素的思路，博弈树的搜素不应该是盲目的在每个分支中完整搜素，当前的状态必须对搜素行为本身有所反馈。例如本程序的 LMR 启发搜素和杀手着法启发，被应用于启发式改进当前的主要变例搜素，提高搜素性能。

最后，PVS 主要变例搜素极其依赖良好的着法排序。否则 PVS 的搜素效率会大幅退化。本程序采用的 MVV/LVA、置换表最佳着法优先、杀手着法优先、安静着法判定以及 LMR 延迟缩减等算法也同样服务于一个目标：让更可能产生 beta 截断的着法尽早被搜索。

4.1.3 程序调试问题

程序在 botzone 上运行，应该如何调试？与算法 OJ 评测不同，本程序的调试不是只打断点就可以完成的——本程序的代码长度极长(4000 行左右)、逻辑分支数目繁多，对于程序在编译时出现的错误的解决相对容易，然而不易解决的是运行时错误和程序内部逻辑错误。上面的错误分别会导致程序可能在 botzone 中输出非法值、程序能够正常运行，但是程序的棋力性能极其差劲，即程序着棋时输出大量极其低劣的棋步。为了解决上面的问题，我们在程序中加入了大量的断

言和异常捕获，增强程序的健壮性，并且能够在程序异常时定位程序错误的原因和位置；并在主程序中加入了输出到 botzone 平台上用于调试、展现当前程序所计算出各个棋步详细信息(得分、着法数、着法分类)的 Json，以便于我们进行进一步调试。此外，我们在部分调试修改的过程中使用了 AI 辅助，在此基础上进一步纠错审查。

4.1.4 数据结构问题

“程序 = 算法 + 数据结构”，Nikilaus Wirth 的名言指明，在程序构建中，数据结构同样起到举足轻重的作用。那么，应该如何使用合理的数据结构？本程序又是如何使用数据结构的？本程序使用哈希表、键值对(C++ STL map 字符到棋子的映射)、队列(C++ STL vector 模拟，如程序中着法排序形成的队列)、栈(C++ STL vector 模拟，如程序中 state 模拟状态栈)，这些结构既让算法易于实现，也提高了程序的性能。

另外，本程序大量使用引用/指针传递形参，在数据结构上避免了大量无用的内存拷贝开销。在构建、遍历“搜素树”时，本程序将着法作为向下搜素的边，(相应的，每次回溯时作为向上返回的反向边)，在单例棋盘状态类的状态栈中运行某一着法和撤销某一着法，使得程序开销极低，不需要每次存储当前棋盘状态造成额外开销。

4.1.5 项目协作构建问题

在编写项目的过程中，如何协调进度？如何对同一工作区内的相同文件进行修改和版本管理？我们使用了 Vscode + GitHub 代码托管的形式，从而同步推进工作流。

并且，在构建整个项目的全部过程中，我们在线下多次交流和研讨，团结协作，解决了诸多问题。可以说，没有良好的协作与开发模式，就没有如今性能和表现良好的项目。

4.2 对本程序的评价与反思

本程序整体上已经形成了较为完整的中国象棋博弈 AI 框架，能够将 Botzone 平台输入的历史走法恢复为当前棋局，再通过合法着法生成、局面价值评估、博弈树搜索和最优着法筛选完成自动走棋决策。程序的优点在于结构分层比较清晰：Chess 类负责棋盘状态、规则判断、合法着法生成和落子回溯；EvalRecord 类集中完成中残局阶段权重、静态局面评估、动态控制力评估、特殊棋型惩罚和着法初步评分；ChessAI 类则负责 PVS、Alpha-Beta、置换表、静态搜索、空步裁剪、LMR、杀手着法、迭代加深和时间控制等搜索优化。这种设计使程序不仅能依靠子力价值判断局面，还能进一步考虑棋子位置、控制力、攻防关系、受威胁状态和特殊危险棋型，因此相比简单的随机走棋或纯子力评估程序，具有更强的实战针对性和更高的搜索效率。尤其是开局库、Zobrist Hash、Transposition Table、Fail-Soft PVS、Quiescence Search 等技术的加入，使程序能够在 Botzone 有限时间内尽可能减少重复搜索、降低无效分支数量，并提高候选着法排序质量，从而更快找到较优走法。

与此同时，程序也存在一些不足和可以改进的地方。首先，评价函数虽然已经综合了多种因素，但各项权重、棋子位置表、动态控制力奖励和特殊棋型惩罚仍主要依赖人工经验调参，这些参数部分来源于人为调整和预测、部分来自于公开的学术资源，但是这缺少大量对局数据或自动化调参结果支撑，并且某些数据已经“过时”了，因此在不同局面下可能出现过度重视某类特征或低估长期战略价值的问题。具体表现为：程序的防守/进攻倾向更加显著、对某些潜在风险并不敏感，或者对某种风险过于敏感。

其次，搜索算法虽然加入了多种剪枝优化，但整体复杂度仍然较高，面对分支数较大的中盘复杂局面时，搜索深度和稳定性仍受时间限制影响明显；空步裁剪、LMR 等激进剪枝技术如果条件设置不够精细，也可能在战术局面中误剪关键变化。具体表现为：如果

最后，当前程序的开局库和评估体系更多服务于规则型搜索，并未结合机器学习、蒙特卡罗搜索或自对弈优化，因此程序棋力提升仍然需要更有效的算法进行改进。

4.3 项目收获与经验教训

在经过三周紧张的设计、编码和调试，以及经过一个五一假期的调整和润色，在此期间，我们收获颇多——和 OJ 算法题目或是先前的课设项目不同，本程序重在算法和数据结构的设计、分析、构建、

优化、调试的全过程。

对我们而言，使用如此繁多的算法，从无到有解决这样一个复杂的问题是前所未有的。我们通过研究网络、学术资源，使用大量的算法完成整个问题的解决过程。诚然，现在的程序的确还不够完美，但我们在解决问题的整个过程，尝试了足够丰富的方法，收获了算法和数据结构分析问题、解决问题的能力，让我们对算法和数据结构有更加深入而全面的认识。

千变万化、思绪飞扬，我想这不仅仅是本次课设项目“中国象棋”的特点，更是计算机博弈的魅力所在。本次课设更让我们入门和初步探索计算机博弈相关的实践和理论，让我们看到抽象的数据结构和算法是如何构筑成计算机博弈程序的血肉和灵魂的；本次课设也激发了我们的求知和探索兴趣，拉近了我们所学理论与现实之间的距离——算法和数据结构不是 OJ 上冰冷的评测试题，而是应用于具体程序之中的引擎；在今天的计算机理论发展中，算法和数据结构更是构成高性能计算方法的根基。

考虑到自身水平问题、课程设计限制、时间要求，本程序避开机器学习及神经网络相关算法，也没有使用 MCTS。这也是对本次课程设计的一种遗憾。

本项目终于迎来了她的尾声。最后，感谢虞强源老师、朱允刚老师的鼓励和指点、感谢组员之间的一致协作。

5.AI 使用情况

5.1 AI 辅助学习

本项目从开始到完成，使用了 AI 分析和生成的内容进行学习，具体内容为：

使用 AI 辅助讲解其它中国象棋博弈的论文

使用 AI 辅助拆解其它程序的源码和原理，进行学习

使用 AI 辅助润色了本报告的部分内容，在本程序的时间复杂度评估和实际性能分析时，使用 AI 辅助计算和提示。

5.2 AI 辅助优化

本项目在算法实现中使用 AI 进行优化。

具体内容为：

使用 AI 对当前程序的语法错误进行纠错，并使用 AI 的建议作为参考，改正当前程序具有诸如内存安全问题、悬空指针或悬空引用等隐患。

在部分算法的实现过程中，采用了 AI 的建议，在已有代码的基础上对算法进行进一步细致优化，例如 PVS 主要遍历搜素中的零窗口搜素简化等等。

5.3 AI 辅助调试

本程序在程序运行与实践中采用 AI 辅助调试。

具体内容为：

在程序的调试环节，参考 AI 提示在当前程序引入多处异常和异常捕获。并参考 AI 思路和部分提示，在 AI 辅助下增加了在 Botzone 上输出调试信息的语句，完善在 Botzone 中调试内容。

5.4 AI 辅助代码审查

本程序在程序采用 AI 辅助纠错审查。

具体表现为：

在程序运行遇到较为隐蔽问题（如搜索排序异常）且未被人工识别导致程序未能正确将搜索结果排序，在使用 AI 审查代码找出致命问题并辅助修正，解决错误使程序正确运行。

5.5 AI 辅助资料查阅与解析

本程序在早期资料查询与思路构建期间提供辅助搜索与解析

具体表现为：

部分文献与算法思路由 AI 辅助整理并提出构建意见（如在静态搜

索中的不良剪枝相关资料），在 AI 辅助下综合了多种思路并选取优解。

5.6 AI 辅助将程序转换为 rust 语言

结果如下：

程序内容和对局效果基本一致，可认为该 rust 转换后的 bot 具备原有 bot 的几乎全部特性。



6.参考文献

6.1 参考网站链接(为简洁起见，部分多个连载的文章仅粘贴其中一部分的网站域名链接)

https://www.xqbase.com/ 象棋巫师

https://www.xqbase.com/computer/stepbystep1.htm

《电脑象棋循序渐进》教程

https://www.xqbase.com/computer/outline.htm

《对弈程序基本技术》教程

https://www.xqbase.com/resource.htm#resources 中国象棋学术资源

https://gamedev.net/ 全球游戏开发者论坛

https://gamedev.net/tutorials/programming/artificial-intelligence/chess-programming-part-ii-data-structures-r1046/ 象棋游戏程序教程

https://www.cnblogs.com/royhoo/p/6426394.html JavaScript 中国象棋程序构建教程

https://www.cnblogs.com/tuanzang/archive/2013/02/27/2935743.html

中国象棋的某些想法

https://www.cnblogs.com/tuanzang/archive/2013/02/27/2935729.html

中国象棋的部分实现代码

https://oi.wiki/math/game-theory/zero-sum-game/

博弈论与游戏基本理论

https://github.com/xqbase/eleeye 象眼中国象棋

https://www.xqbase.com/league/mrsd.htm 梦入神蛋中国象棋

6.2 参考文献资料

1.《利用统计数据并行蒙特卡罗树搜索算法的中国象棋博弈》，朱舟、闵华松，《Computer Engineering and Applications》，2024，60(23)

p.s: 事实上，考虑到使用 MCTS 算法，需要构建神经网络设计、进行大量网络训练得到。考虑到自身水平问题、课程设计限制、时间要求，本程序避开机器学习及神经网络相关算法，也没有使用 MCTS。这也是对本次课程设计的一种遗憾。

2.《计算机博弈的研究与发展》，沈阳航空航天大学工程训练中心， 王亚杰，邱虹坤，吴燕燕，李飞，杨周凤，《智能系统学报》，2016

3.《人与机器人象棋对弈系统的设计与实现》，中北大学，硕士学位论文，韩燮、田卫萍，2019

4.《中国象棋博弈搜素算法研究与实现》，沈阳工业大学，硕士学位论文，宋兴亮，2012.02.23

5.《中国象棋博弈——局面评估研究》，南昌大学，硕士学位论文，罗涛、白似雪，2009.12.27

6.《中国象棋博弈系统的研究与实现_危春波》，昆明理工大学，硕士学位论文，危春波、王海瑞，2008.3.5

7.《中国象棋计算机博弈数据结构与评估函数的研究和实现》，西安理工大学，硕士学位论文，谢艳茹、钱富才，2008.3